

esptool.py

Wrapper Script for ROM Bootloader

The esptool.py script is a wrapper script for the ESP32/ESP8266 ROM bootloader utility called esptool. This document explains the tool as well as the interaction between the MAC/PC and the bootloader on the ESP32, along with tips on how to interpret traces using an SALEAE Original Logic Analyzer.

Purpose of the ESPTOOL.PY script

- esptool.py is a command-line utility developed by Espressif Systems that allows you to communicate with the ROM bootloader in ESP32 and ESP8266 chips
- It's used for flashing firmware, reading/writing flash memory, and performing various low-level operations on ESP chips

ESPTOOL.PY Script Structure

- This particular file is a thin wrapper around the main esptool Python package
- It's designed to ensure proper module loading in different operating systems

Key Features

- Path Management: The script handles different path configurations for Windows vs Linux/macOS
- For non-Windows systems (Linux/macOS), it:
 - Removes the current script directory from Python's path to avoid importing this file as a module
 - Ensures it imports the installed esptool module instead
 - Deletes any previously imported module entry to force Python to load the module fresh

Usage Context

This tool is crucial in the ESP32 development workflow, especially when:

- Flashing firmware to your ESP32 board
- Reading flash contents
- Verifying firmware
- Performing chip identification
- Managing boot parameters

Integration

In your PlatformIO project (esp32DevBoard), this tool is used automatically during:

- Firmware uploads
- Flash operations
- Board communication during programming

The main functionality isn't in this wrapper script but in the esptool package it imports. This wrapper ensures proper execution of the tool across different operating systems and environments. When you use PlatformIO to upload firmware to your ESP32 board, it uses this tool behind the scenes to handle the actual communication with the board's bootloader.

Bootloader handshake

The bootloader handshake between esptool and the ESP32 works as follows:

Initial Setup:

- The tool opens a serial connection to the ESP32 at 115200 baud (default bootloader baud rate)
- The ESP32 must be in bootloader mode, which happens when:
 - GPIO0 is held low during reset
 - Or when the chip is reset while the UART RTS and DTR lines are toggled in a specific pattern

Reset Sequence:

- The tool toggles the DTR and RTS lines to force the ESP32 into bootloader mode:
 1. Set DTR low, RTS high (pulls both EN and GPIO0 low)
 2. Brief delay
 3. Set DTR high, RTS low (releases EN while keeping GPIO0 low)
 4. This causes a reset while GPIO0 is held low, entering bootloader mode

Synchronization:

- The tool sends a synchronization pattern:
 - 0x07 0x07 0x12 0x20 followed by 32 more bytes
 - This specific pattern helps the ROM bootloader detect the correct baud rate
- The ESP32 responds with:
 - 0xC0 (indicating successful sync)
 - Then sends information about its capabilities

Handshake Completion:

- After sync, the tool can:
 - Change the baud rate to a higher speed
 - Start sending commands to read/write flash
 - Get chip information
 - Etc.

Error Handling:

If sync fails, the tool will:

- Retry multiple times
- Try different baud rates
- Attempt different reset timing patterns

This makes the process robust against varying hardware configurations

Key Commands After Sync:

- READ_REG: Read memory registers
- FLASH_BEGIN: Start flash operation
- FLASH_DATA: Send flash data
- FLASH_END: End flash operation
- MEM_BEGIN: Start RAM operation
- MEM_DATA: Send RAM data
- MEM_END: End RAM operation

The handshake process ensures reliable communication is established before any flash operations begin. It's designed to be robust enough to work across different ESP32 variants and board designs.

This is a critical part of the firmware update process, as it ensures the ESP32 is in a known state before any flash operations begin. The process is handled automatically by esptool.py when you upload new firmware through PlatformIO or the Arduino IDE.

Using a logic analyser

The D+ and D- signals you're seeing are USB signals between your Mac and the CP2102 USB-to-UART bridge chip. However, the actual bootloader handshake between the CP2102 and ESP32 happens over UART (serial) lines. You'll want to probe different signals to see the bootloader handshake.

Here's what you should probe:

- For USB Analysis (Mac to CP2102):
 - D+ and D- lines

- Set probe to USB Full Speed (12 Mbps)
- This will show USB packets but isn't the bootloader protocol
- For Bootloader Handshake (CP2102 to ESP32):
 - TX (transmit) from CP2102 to ESP32's RX
 - RX (receive) from ESP32's TX to CP2102
 - RTS (Request to Send) line
 - DTR (Data Terminal Ready) line
 - Set probe to:
 - Protocol: UART/Serial
 - Baud Rate: 115200 (initial rate)
 - Data Bits: 8
 - Stop Bits: 1
 - Parity: None
 - Flow Control: None

The actual bootloader handshake happens on the UART lines, not the USB lines. The sequence you want to capture is:

- Reset sequence (watch RTS and DTR lines)
- Sync pattern (0x07 0x07 0x12 0x20 + 32 bytes on TX)
- Response from ESP32 (0xC0 + capability bytes on RX)

USB errors we have been seeing

The USB errors you're seeing are likely normal USB protocol retries and aren't directly related to the ESP32 bootloader handshake protocol. You should focus on the UART signals after the CP2102 for analyzing the bootloader communication.

Set up the Saleae Original Logic Analyzer

We have been using the Saleae Original Logic Analyzer with software version 1.2.40 to sniff the interaction between the Mac/PC and the ESP32.

For the Saleae Logic 1.2.40 software, you'll want to set up two different protocol analyzers:

1. For USB Analysis (Optional, if you want to see Mac to CP2102):
 - Add Async Serial analyzer
 - Add USB Low/Full Speed analyzer
 - Set Speed to: Full Speed (12 Mbps)
 - Connect to D+ and D- pins
2. For ESP32 Bootloader Communication (This is what you really want):
 - Add Async Serial analyzer
 - Configure as follows:

- Bit Rate: 115200
- Bits per Transfer: 8
- Stop Bits: 1
- Parity Bit: None
- Significant Bit: Least First
- Idle State: High

Channel Setup:

1. Channel 0: ESP32 RX (CP2102 TX)
2. Channel 1: ESP32 TX (CP2102 RX)
3. Channel 2: RTS
4. Channel 3: DTR

Trigger Setup:

To catch the start of the bootloader sequence, trigger on the RTS/DTR reset pattern:

- Set trigger on both RTS and DTR changing state
- This will catch the reset sequence that initiates bootloader mode

Recommended Capture Settings:

- Sample Rate: At least 16MHz (higher is better)
- Duration: At least 1 second to catch the full handshake
- Buffer Size: Set to maximum available

Tips for Saleae Logic specifically:

1. Use the "Timing Markers Pair" feature to measure the reset pulse widths
2. Use the "Async Serial" analyzer's "Data Export" to see the hex values
3. Look for these specific patterns:
 - Reset sequence: DTR and RTS toggling
 - Sync pattern: 0x07 0x07 0x12 0x20 followed by 32 bytes
 - ESP32 response: 0xC0 followed by capability bytes

To decode the data:

- Right-click on the Async Serial analyzer
- Select "Export decoded protocol data"
- Choose "CSV" format
- Look for the specific byte patterns mentioned above

If you're seeing garbled data:

- Double-check ground connection
- Verify voltage levels (Saleae Logic works with 3.3V and 5V)
- Try increasing the sample rate
- Make sure probe leads are kept short to minimize noise

Specific patterns to look for

Let me break down the specific patterns you should look for in your Saleae Logic capture during the ESP32 bootloader handshake sequence:

1. Reset Sequence (DTR/RTS lines):
 - Initial state: DTR=1, RTS=1
 - Reset start: DTR=0, RTS=1 (pulls both EN and GPIO0 low)
 - Boot mode: DTR=1, RTS=0 (releases EN, keeps GPIO0 low)
 - ***Look for this pattern happening in < 50ms***
2. Sync Pattern (on TX line from Mac/PC to ESP32):
 - Header: 0x07 0x07 0x12 0x20
 - Payload: 32 bytes of 0x55
 - ***This is the initial synchronization attempt. You might see it repeated if first try fails.***
3. ESP32 Response (on RX line from ESP32 to Mac/PC):
 - Success: 0xC0 (first byte)
 - Following: 0x00 0x08 (response length)
 - Then: 0x24 0x00 0x00 0x00 0x00 0x00 0x00 0x00 (capability bits)
4. Command Sequences (after sync):
 - Command headers start with: 0xC0 or 0x00
 - Common commands:
 - 0x08 (READ_REG)
 - 0x03 (FLASH_BEGIN)
 - 0x02 (FLASH_DATA)
 - 0x04 (FLASH_END)
5. Timing Patterns:
 - Reset pulse width: ~10ms
 - Time between reset and first sync attempt: ~50ms
 - Response timeout: ~100ms
 - Inter-byte spacing at 115200 baud: ~87µs
6. Error Patterns: If sync fails, look for:
 - Repeated sync patterns (0x07 0x07 0x12 0x20)
 - Varying delays between attempts
 - No 0xC0 response from ESP32
7. In the Saleae Logic software:
 - Use the protocol analyzer's search function to find 0x07 0x07 sequence
 - Place timing markers at the DTR/RTS transitions
 - Look for the 0xC0 response in the RX data
 - Export decoded data to CSV and search for these patterns:
8. Search terms:

- "07 07 12 20" (sync request)
- "C0 00 08" (successful response start)
- "03" (FLASH_BEGIN command)

The USB side of the conversation

This section explains the USB-side communication between the Mac/PC and the CP2102 chip.

Initial USB Enumeration

When you connect the ESP32 development board, this sequence occurs:

1. USB Device Detection
 - Mac detects 5V on USB bus
 - Device responds with USB 2.0 full-speed (12Mbps) capability
2. Device Identification
 - VID: 10C4 (Silicon Labs)
 - PID: EA60 (CP210x UART Bridge)
3. Driver Loading
 - MacOS loads the built-in CP210x driver
 - Creates virtual serial port (typically /dev/tty.SLAB_USBtoUART)

USB Descriptor Exchange

The Mac requests these descriptors:

1. Device Descriptor
 - USB version
 - Device class (CDC - Communications Device Class)
 - Maximum packet size
2. Configuration Descriptor
 - Power requirements
 - Interface descriptions
 - Endpoint configurations

Monitoring the USB Traffic

You can observe this using these tools:

1. System Report
 - `system_profiler SPUSBDataType`
2. USB Prober (if you have Apple Developer Tools)
3. Wireshark with USB Capture
 - `sudo wireshark -i usb0`

Key Control Transfers

During esptool.py operation:

1. SET_LINE_CODING (0x20)
 - Baud rate configuration
 - Data bits, stop bits, parity
2. SET_CONTROL_LINE_STATE (0x22)
 - DTR/RTS signal control for ESP32 reset/boot sequence

These USB-level operations happen before the UART communication begins with the ESP32. Understanding this helps when debugging connection issues between your Mac and the ESP32.